

Task 1:

```
env = gym.make("FrozenLake-v1", map_name="4x4",  
is_slippery=False)
```

```
In [13]: import gymnasium as gym  
from stable_baselines3 import DQN  
from stable_baselines3.common.evaluation import evaluate_policy  
  
# --- Build environment ---  
env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=False)  
print("Env. Action space: ", env.action_space)  
print("Env. observation space: ", env.observation_space)  
  
# --- Build, Train and Evaluate Model ---  
model_ns = DQN(  
    "MlpPolicy",  
    env,  
    learning_rate=1e-3,  
    buffer_size=10000,  
    learning_starts=1000,  
    batch_size=64,  
    gamma=0.99,  
    train_freq=1,  
    target_update_interval=250,  
    exploration_fraction=0.3,  
    exploration_final_eps=0.05,  
    verbose=0,  
)  
model_ns.learn(total_timesteps=100_000)  
mean_reward, _ = evaluate_policy(model_ns, model_ns.get_env(), n_eval_ep  
  
# --- Results ---  
print(f"\nMean reward over 100 episodes: {mean_reward:.2f}")
```

```
Env. Action space: Discrete(4)  
Env. observation space: Discrete(16)
```

```
Mean reward over 100 episodes: 1.00
```

```
In [ ]: # --- Create a new renderable env just for testing/visualization ---  
render_env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=False)  
  
obs, _ = render_env.reset()  
done = False  
while not done:  
    action, _ = model_ns.predict(obs, deterministic=True)  
    action = int(action)  
    obs, reward, terminated, truncated, _ = render_env.step(action)  
    done = terminated or truncated
```

Task 2:

```
env = gym.make("FrozenLake-v1", map_name="4x4",  
is_slippery=True)
```

In [14]:

```
import gymnasium as gym  
from stable_baselines3 import DQN  
from stable_baselines3.common.evaluation import evaluate_policy  
  
# --- Build environment ---  
env.reset()  
env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=True)  
print("Env. Action space: ", env.action_space)  
print("Env. observation space: ", env.observation_space)  
  
# --- Build, Train and Evaluate Model ---  
model_s = DQN(  
    "MlpPolicy",  
    env,  
    learning_rate=1e-3,  
    buffer_size=10000,  
    learning_starts=1000,  
    batch_size=128,  
    gamma=0.99,  
    train_freq=1,  
    target_update_interval=200,  
    exploration_fraction=0.3,  
    exploration_final_eps=0.05,  
    verbose=0,  
)  
model_s.learn(total_timesteps=100_000)  
mean_reward, _ = evaluate_policy(model_s, model_s.get_env(), n_eval_episodes=100)  
  
# --- Results ---  
print(f"\nMean reward over 100 episodes: {mean_reward:.2f}")
```

```
Env. Action space: Discrete(4)  
Env. observation space: Discrete(16)
```

```
Mean reward over 100 episodes: 0.77
```

In [5]:

```
# --- Create a new renderable env just for testing/visualization ---  
render_env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=True,  
render_mode="human")  
  
obs, _ = render_env.reset()  
done = False  
while not done:  
    action, _ = model_s.predict(obs, deterministic=True)  
    action = int(action)  
    obs, reward, terminated, truncated, _ = render_env.step(action)  
    done = terminated or truncated
```

```
2025-03-27 21:32:30.746 Python[84495:18596685] +[IMKClient subclass]: chose IMKClient_Modern
2025-03-27 21:32:30.746 Python[84495:18596685] +[IMKInputSession subclass]: chose IMKInputSession_Modern
```

Task 3:

```
env = gym.make("FrozenLake-v1", map_name="8x8",
is_slippery=True)
```

```
In [17]: # --- Build enviornment ---
env.reset()
env = gym.make("FrozenLake-v1", map_name="8x8", is_slippery=True)
print("Env. Action space: ", env.action_space)
print("Env. observation space: ", env.observation_space)

# --- Build, Train and Evaluate Model ---
model = DQN(
    "MlpPolicy",
    env,
    learning_rate=1e-3,
    buffer_size=10000,
    learning_starts=1000,
    batch_size=128,
    gamma=0.99,
    train_freq=1,
    target_update_interval=200,
    exploration_fraction=0.3,
    exploration_final_eps=0.05,
    verbose=0,
)
model.learn(total_timesteps=100_000)
mean_reward, std_reward = evaluate_policy(model, model.get_env(), n_eval

# --- Results ---
print(f"\nMean reward over 200 episodes: {mean_reward:.2f} ± {std_reward
```

```
Env. Action space: Discrete(4)
Env. observation space: Discrete(64)
```

```
Mean reward over 200 episodes: 0.55 ± 0.50
```

```
In [11]: # --- Create a new renderable env just for testing/visualization ----
render_env = gym.make("FrozenLake-v1", map_name="8x8", is_slippery=True,

obs, _ = render_env.reset()
done = False
for i in range(200):
    action, _ = model.predict(obs, deterministic=True)
    action = int(action)
    obs, reward, terminated, truncated, _ = render_env.step(action)
    done = terminated or truncated
    if done:
        break
```

Comparison

In [12]:

```
def get_action(model, state_idx):
    obs = state_idx
    action, _ = model.predict(obs, deterministic=True)
    action = int(action)
    return action

# --- For is_slippery=False ---

print(" --- is_slippery = False ---")
print(f'Preferred action in state 1: {get_action(model_ns, 1)}')
print(f'Preferred action in state 6: {get_action(model_ns, 6)}')

# --- For is_slippery=True ---

print(" --- is_slippery = False ---")
print(f'Preferred action in state 1: {get_action(model_s, 1)}')
print(f'Preferred action in state 6: {get_action(model_s, 6)}')
```

```
--- is_slippery = False ---
Preferred action in state 1: 2
Preferred action in state 6: 1
--- is_slippery = False ---
Preferred action in state 1: 3
Preferred action in state 6: 0
```

Reason for behaviour

In the deterministic environment, there is no risk involved in choosing a path, leading to the optimal path being chosen every time. However in the stochastic environment, a risk is involved in being surrounded by holes, leading to avoidance of areas like state 6 all together. The stochastic environment is choosing against the optimal path, due to an increased risk of falling into a hole.